

APRENDIZADO SUPERVISIONADO

EAB067 - Aprendizado de Máquina

Atualizado em: 5 de maio de 2026

Iago Carvalho

Departamento de Ciências da Computação

Programa de Pós-Graduação em
Estatística Aplicada e Biometria



APRENDIZADO SUPERVISIONADO

Como vimos nas aulas anteriores, algoritmos de aprendizado de máquina supervisionados utilizam dados **rotulados**

Além disso, também vimos que suas principais aplicações são

- Regressão
- Classificação

Hoje vamos discutir a teoria por trás do aprendizado supervisionado e os principais algoritmos de aprendizado supervisionado

- Regressão logística
- KNN
- Árvores de classificação e regressão (CART)
- Redes neurais

FUNDAMENTOS DA TEORIA DO APRENDIZADO

Definição Formal

Dado um conjunto de treinamento $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, gerado i.i.d. por uma distribuição de probabilidade conjunta desconhecida $P(x, y)$, o objetivo é induzir uma função $f : \mathcal{X} \rightarrow \mathcal{Y}$ a partir de um espaço de hipóteses $\mathcal{H}$.

Componentes:

- $x \in \mathbb{R}^d$: Vetor de características (Features).
- $y \in \mathcal{Y}$: Variável alvo (Target).
- Se $\mathcal{Y} = \mathbb{R} \implies$ **Regressão**.
- Se $\mathcal{Y} = \{c_1, c_2, \dots, c_k\} \implies$ **Classificação**

RISCO ESPERADO VS. RISCO EMPÍRICO

A qualidade de f é medida por uma função de perda $L(y, f(x))$

- y é o dado verdadeiro
- $f(x)$ é o valor estimado

O **Risco Esperado** (Erro de Generalização) é a esperança da perda em toda a distribuição P

$$R(f) = \mathbb{E}_{(x,y) \sim P}[L(y, f(x))]$$

Como P é desconhecida, o problema de otimização exato é intratável

Recorremos à **Minimização do Risco Empírico (ERM)**

$$\hat{R}(f) = \frac{1}{N} \sum_{i=1}^N L(y^{(i)}, f(x^{(i)}))$$

A HIPÓTESE LINEAR

Assumimos que a relação entre as características e o alvo é aproximadamente linear

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d \quad \leftarrow \quad \text{Regressão linear}$$

Podemos usar a notação vetorial compacta

$$h_{\theta}(x) = \theta^T x$$

onde $\theta \in \mathbb{R}^{d+1}$ e $x \in \mathbb{R}^{d+1}$.

DERIVAÇÃO PROBABILÍSTICA DA MÁXIMA VEROSSIMILHANÇA

Por que usar o Erro Quadrático Médio? Assuma que as previsões são corrompidas por um ruído Gaussiano $\epsilon \sim \mathcal{N}(0, \sigma^2)$:

$$y^{(i)} = \underbrace{\theta^T x^{(i)}}_{\text{Constante}} + \underbrace{\epsilon^{(i)}}_{\text{Ruído Gaussiano}}$$

A probabilidade de observar os dados dados os parâmetros é:

$$P(y|X; \theta) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Maximizar a log-verossimilhança (MLE) é matematicamente equivalente a **minimizar a soma dos erros quadráticos**.

REGRESSÃO LOGÍSTICA E CLASSIFICAÇÃO LINEAR

DA REGRESSÃO PARA A CLASSIFICAÇÃO

Por que não usar Regressão Linear Múltipla para classificação binária ($y \in \{0, 1\}$)?

- As previsões de um modelo linear $h_{\theta}(x) = \theta^T x$ podem assumir valores em $(-\infty, +\infty)$, perdendo o sentido probabilístico.
- Modelos estritamente lineares são sensíveis a *outliers*, que podem deslocar a fronteira de decisão de maneira indesejada.

Desta forma, deve-se escolher uma função que mapeia uma saída linear no domínio $[0, 1]$.

A FUNÇÃO SIGMOIDE (LOGIT)

Introduzimos a função logística (ou sigmoide):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

A nossa hipótese passa a ser uma probabilidade condicional:

$$h_{\theta}(x) = P(y = 1|x; \theta) = \sigma(\theta^T x) = \frac{1}{1 + \exp(-\theta^T x)}$$

Propriedade fundamental da derivada:

$$\frac{d}{dz}\sigma(z) = \sigma(z)(1 - \sigma(z))$$

Com esta propriedade é fácil implementar métodos de descida de gradiente para resolver o problema de otimização convexa associado

Como tomamos a decisão final de classificação?

Define-se um limiar (*threshold*), tipicamente 0,5.

$$\hat{y} = \begin{cases} 1 & \text{se } h_{\theta}(x) \geq 0,5 \\ 0 & \text{se } h_{\theta}(x) < 0,5 \end{cases}$$

K-NEAREST NEIGHBORS (K-NN)

O PARADIGMA NÃO-PARAMÉTRICO

Os modelos anteriores (Regressão e Logística) são **paramétricos**: assumem uma forma funcional forte (θ) e comprimem os dados nesses parâmetros.

O K-Nearest Neighbors (K-NN) é:

- **Não-paramétrico**: Não assume uma distribuição rígida subjacente aos dados
 - Cresce em complexidade com a base de dados
- **Baseado em Instâncias (*Lazy Learning*)**:
 - Não há uma fase explícita de *treinamento*
 - A função de hipótese é computada localmente apenas no momento da inferência

ESPAÇOS MÉTRICOS E FUNÇÕES DE DISTÂNCIA

O K-NN assume que instâncias com características similares (*features*) devem ter saídas similares (*targets*). A noção de "similaridade" exige uma métrica.

Distância de Minkowski (Norma L_p):

$$d_p(x_i, x_j) = \left(\sum_{m=1}^d |x_{im} - x_{jm}|^p \right)^{\frac{1}{p}}$$

- $p = 1$ (**Manhattan**): Robusta a *outliers*. Geometria de grade.
- $p = 2$ (**Euclidiana**): O padrão em espaços reais. Invariante à rotação, mas requer que as features sejam escalonadas (normalização/padronização).

A REGRA DE DECISÃO DO K-NN

Seja $\mathcal{D}$ o conjunto de treino e $\mathbf{x}_q$ uma nova consulta (*query*). Definimos $\mathcal{N}_k(\mathbf{x}_q) \subseteq \mathcal{D}$ como o conjunto dos k vizinhos mais próximos segundo uma distância métrica d .

Para Classificação (Votação Majoritária):

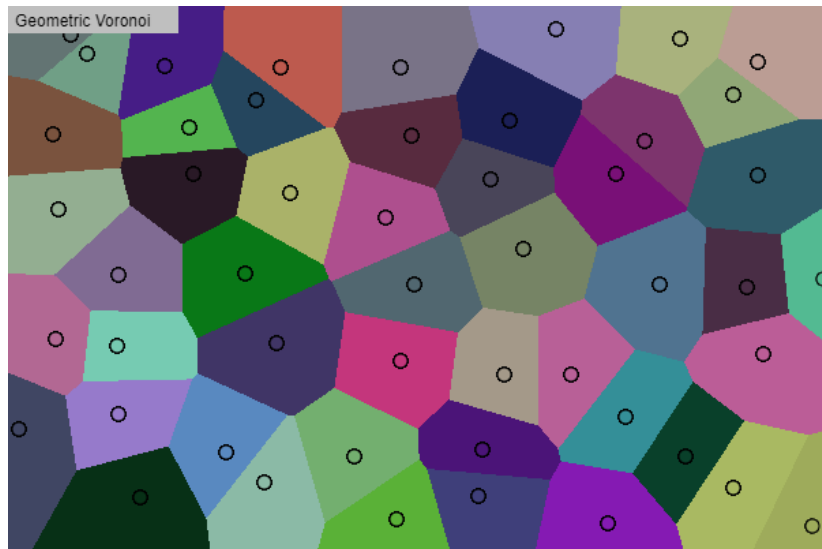
$$\hat{y}_q = \arg \max_{c \in \mathcal{Y}} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{N}_k(\mathbf{x}_q)} \mathbb{I}(y_i = c)$$

Para Regressão (Média Local):

$$\hat{y}_q = \frac{1}{k} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{N}_k(\mathbf{x}_q)} y_i$$

- Quando $k = 1$, o K-NN particiona o espaço de características $\mathcal{X}$ em um **Diagrama de Voronoi**.
- Cada ponto de treinamento x_i domina um polígono complexo onde qualquer ponto interno está mais próximo a ele do que de qualquer outro ponto.
- A fronteira de decisão global resultante é altamente não-linear e fragmentada.

VORONOI COM $k = 1$



O HIPERPARÂMETRO k E O TRADE-OFF VIÉS-VARIÂNCIA

A escolha de k controla a flexibilidade do modelo:

- k muito pequeno (ex: $k = 1$):
 - Modelo extremamente flexível.
 - **Alta Variância / Baixo Viés:** Captura o ruído local perfeitamente (*overfitting*). Fronteiras denteadas.
- k muito grande (ex: $k = N$):
 - Modelo extremamente rígido.
 - **Baixa Variância / Alto Viés:** Prediz sempre a média global ou a classe majoritária (*underfitting*). Fronteiras suaves.

A MALDIÇÃO DA DIMENSIONALIDADE E EFICIÊNCIA

O K-NN sofre de duas fragilidades severas em sistemas reais:

1. A Maldição da Dimensionalidade: Em espaços de alta dimensão (d grande), o volume do espaço cresce exponencialmente. Os dados tornam-se esparsos e todos os pontos tendem a estar aproximadamente à mesma distância uns dos outros. A intuição de "vizinhança" colapsa.

2. Complexidade Computacional: A inferência *naïve* tem complexidade $\mathcal{O}(Nd)$, o que é proibitivo. Na prática, utilizamos estruturas de dados de particionamento espacial em árvore para buscas em tempo logarítmico:

- **K-D Trees** (Eficientes para $d < 20$)
- **Ball Trees** (Eficientes para maiores dimensões)

ÁRVORES DE DECISÃO

Modelos lineares, logísticos e baseados em distância sofrem quando as características têm escalas muito diferentes ou relações altamente não-lineares.

Árvores de Decisão:

- Particionam o espaço de características iterativamente com partições ortogonais aos eixos
- Cada nó interno testa uma característica
 - Por exemplo, $x_j \leq s$ ou $x_j > s$
- Cada folha atribui uma classe ou um valor real

OTIMIZAÇÃO COMBINATÓRIA E O ALGORITMO CART

Encontrar a árvore ótima (menor árvore consistente com os dados) é um problema **NP-Completo**

Utilizamos heurísticas **Gulosas (Top-Down)** como o CART (Classification and Regression Trees)

- Em cada nó m , buscamos o par (característica j , ponto de corte s) que minimiza o custo das duas partições resultantes (R_1 e R_2)

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} L(y_i, c_1) + \min_{c_2} \sum_{x_i \in R_2(j,s)} L(y_i, c_2) \right]$$

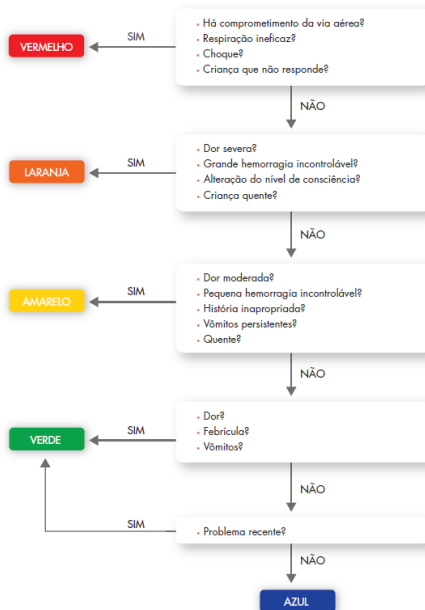
FUNCIONAMENTO DE ÁRVORES DE DECISÃO

Na prática, uma árvore de decisão funciona muito parecido com a mente humana

- Por isso, ela cria modelos de aprendizado de máquina altamente explicáveis

Modelos de árvore de decisão podem ser impressos e explicados para qualquer pessoa

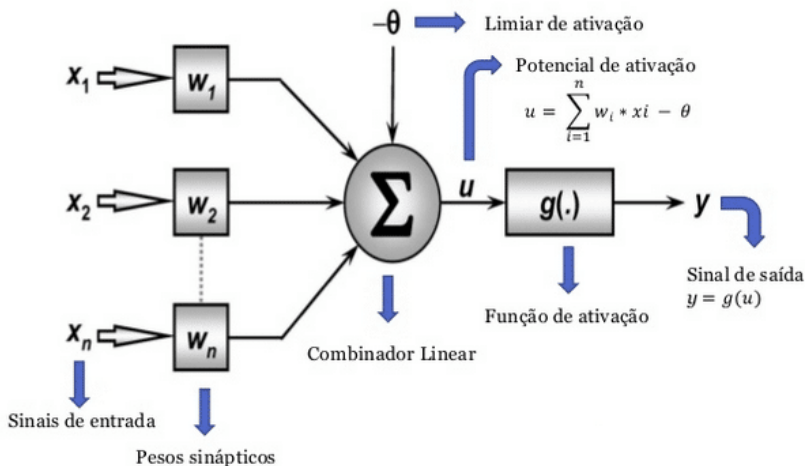
ÁRVORE DE DECISÃO NO CONTEXTO HOSPITALAR



REDES NEURAIS ARTIFICIAIS (MULTILAYER PERCEPTRON)

PERCEPTRON

Redes neurais artificiais são baseadas em **perceptrons** de Rosenblatt



PERCEPTRON

- Entradas (x): O perceptron recebe múltiplos sinais de entrada ($x_1, x_2, \dots, x_n$). Estes são os dados ou características que ele está analisando
- Pesos (w): Cada entrada possui um peso correspondente ($w_1, w_2, \dots, w_n$). Os pesos determinam o grau de importância de cada entrada. Se uma entrada for muito importante para a decisão, seu peso será alto
- Viés (b): O viés (ou *bias*) é um valor extra adicionado à rede. Ele funciona como uma "inclinação" ou ajuste de limiar, permitindo que o perceptron mova a fronteira de decisão para cima ou para baixo, mesmo se todas as entradas forem zero
- Soma Ponderada (z): O perceptron multiplica cada entrada pelo seu respectivo peso e soma todos os resultados, adicionando o viés no final

$$z = \sum_{i=1}^n w_i x_i + b$$

PERCEPTRON

- Função de Ativação (g): O resultado da soma ponderada (z) passa por uma função de ativação, que decide o que o perceptron vai "cuspir" como resultado final. No perceptron clássico, utiliza-se a função degrau (Heaviside)
 - Se $z \geq 0$, a saída é 1 (ex: "Sim", "Aprovado", "Classe A")
 - Se $z < 0$, a saída é 0 ou -1 (ex: "Não", "Reprovado", "Classe B")
- Saída (y): É a decisão final do algoritmo

O perceptron de camada única só consegue aprender padrões que são linearmente separáveis

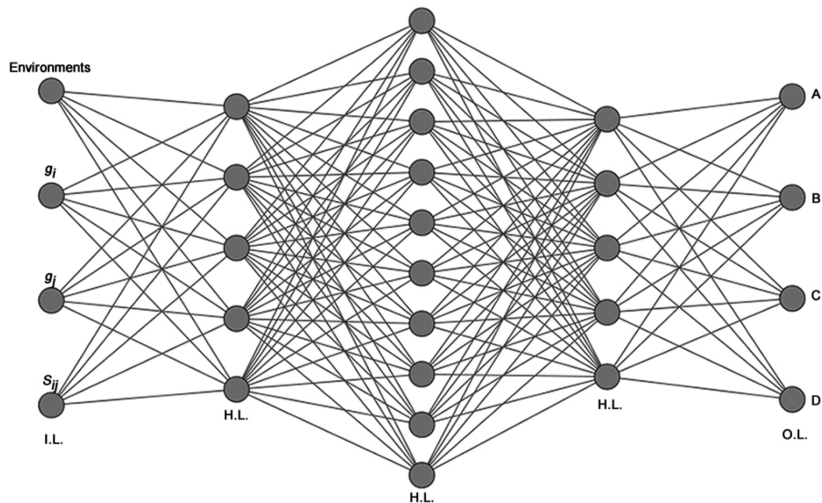
- Isso significa que ele só funciona se você conseguir desenhar uma única linha reta num gráfico para separar perfeitamente a Classe A da Classe B

A solução para isto é criar uma rede com múltiplos perceptrons

- Múltiplas camadas
- Cada camada com sua característica

O resultado disso é um **Multi-Layer Perceptron**

MULTI-LAYER PERCEPTRON



FORMULAÇÃO MATRICIAL DE UMA CAMADA OCULTA

Seja uma entrada $a^{(0)} = x \in \mathbb{R}^d$. Uma camada oculta l realiza duas operações matriciais:

1. Transformação Linear (Afinidade):

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

Onde $W^{(l)}$ é a matriz de pesos e $b^{(l)}$ o vetor de vieses.

2. Transformação Não-Linear (Ativação):

$$a^{(l)} = g(z^{(l)})$$

Onde $g(\cdot)$ é aplicada elemento a elemento

- Sem a não-linearidade, múltiplas camadas colapsariam em uma única camada linear

Uma rede de L camadas é apenas uma função composta parametrizada Θ :

$$f_{\Theta}(x) = a^{(L)} = g^{(L)}(W^{(L)}g^{(L-1)}(\dots g^{(1)}(W^{(1)}x + b^{(1)})\dots) + b^{(L)})$$

Teorema da Aproximação Universal (Cybenko, 1989)

Uma rede neural *feed-forward* com apenas uma única camada oculta contendo um número finito de neurônios pode aproximar qualquer função contínua em subconjuntos compactos de $\mathbb{R}^n$, sob suposições leves sobre a função de ativação.

MÉTRICAS DE AVALIAÇÃO DE MODELOS

MÉTRICAS DE REGRESSÃO

Medem o quão distantes as predições contínuas estão dos valores reais. Seja $\hat{y}_i$ a predição e $\bar{y}$ a média do alvo:

- **MAE (Erro Absoluto Médio):** Robusto a outliers.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **RMSE (Raiz do Erro Quadrático Médio):** Penaliza erros grandes severamente.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

- **R^2 (Coeficiente de Determinação):** Proporção da variância explicada pelo modelo.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

A **Acurácia** mede a taxa de acertos brutos.

$$\text{Acurácia} = \frac{\text{Previsões Corretas}}{\text{Total de Previsões}}$$

O Problema do Desbalanceamento: Imagine classificar uma doença que ocorre em 1 a cada 1000 pacientes. Um modelo estúpido que "sempre preveja Saudável" terá Acurácia de 99.9%, mas **zero utilidade**.

Precisamos inspecionar os tipos de erros.

A MATRIZ DE CONFUSÃO

Mapeia previsões do modelo contra os rótulos verdadeiros.

	Predição: Positivo	Predição: Negativo
Real: Positivo	Verdadeiro Positivo (TP)	Falso Negativo (FN)
Real: Negativo	Falso Positivo (FP)	Verdadeiro Negativo (TN)

- **Erro Tipo I (FP):** Falso Alarme (ex: Spam falso).
- **Erro Tipo II (FN):** Falha de detecção (ex: Deixar passar fraude).

MÉTRICAS DERIVADAS DA MATRIZ

- **Precisão (Precision):** De todas as instâncias que o modelo classificou como positivas, quantas realmente eram?

$$P = \frac{TP}{TP + FP}$$

- **Revocação (Recall / Sensibilidade):** De todas as instâncias que eram de fato positivas, quantas o modelo conseguiu capturar?

$$R = \frac{TP}{TP + FN}$$

- **F1-Score:** O compromisso (Média Harmônica). Penaliza desbalanços severos entre Precisão e Revocação.

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$